OG Code Companion

Technical specification | Version 2.4.0 | 19/10/2026

Purpose

A Windows coding workbench for learners to discover libraries, inspect source examples, understand their syntax and assemble editable programs one addition at a time. Online library access does not require an AI model. Custom model transformations are optional.

Desktop and source

Portable Windows 64-bit application using Python, PySide6 / Qt 6.10.1 and PyInstaller 6.17.0. Build code, Browse libraries and Models / API remain the main tabs. Code, How to build it, Snippet preview and Run output are workbench views. The supplied OG PNG and ICO provide the circular background and application icon.

Source components

og_companion.py implements the Qt window, workers, editing, preview, project files and local process control. og_companion_search.py provides registry discovery, catalogue ranking and local-model validation. og_companion_sources.py retrieves and caches public source text, extracts code blocks and discovers language-compatible examples. og_companion_workbench.py provides source-grounded explanations, language comparisons, syntax diagnostics and argument-vector runtime plans. og_companion_library.json holds editable teaching examples and foundations.

Package

OG Code Companion.exe, Start OG Code Companion.cmd, editable modules/tests, catalogue, assets, runtime/source licences, Read Me and this specification belong in the same folder. The desktop runtime is bundled; executing learner programs requires their separate language tools and dependencies. Build with Build OG Code Companion.ps1. The package name and executable metadata use OG Code Companion.

Scope of evidence

Source checks on 19/10/2026 passed 136 tests, including the original wrong-language scenario, preview preservation, successive additions, undo, project round-trip, real Python execution, runtime errors and installed Node.js syntax diagnostics. Packaged checks are recorded in the build installation-check.json. A source test is distinct from a packaged check, visible interaction or acceptance on another computer.

Library and documentation access

Discovery independent of AI

Registry adapters

Python searches the PyPI name index and standard-library module names. JavaScript/TypeScript use npm; C#/F# use NuGet; Java/Kotlin/Scala use Maven Central; Rust uses crates.io; PHP uses Packagist; Ruby uses RubyGems. Other choices can search public GitHub repositories. Source descriptions, repository README files and public documentation supply examples. Provider availability, authentication limits and rate limits apply.

Documentation and paging

The bundled manifest lists 244 documentation titles across 837 editions. Select an edition and search its topic index, or browse All API topics. Previous and Next reach later pages. Complete collection retrieves every distinct extracted block in the selected edition; the normal task search explores ranked topics and a limited number of relevant sources. Find more snippets is another search, not an assertion that every online source has been exhausted.

Language integrity

A task is matched to documentation within a recognised language family. The word click in a JavaScript button request cannot choose the Python Click collection. Preview lists and additions enforce the selected language; configuration, shell commands, transcripts and other-language blocks are omitted rather than silently loaded as the requested language. Language metadata can still depend on publisher hints and inference, so inspect unfamiliar source.

Measured content

The local teaching catalogue contains 55 snippets across 22 languages and two complete Python foundations. The selector offers 22 language/tool choices. Fourteen supplied documentation collections contain 84,512 summed source blocks; commands, configuration and transcripts are included in that source-block inventory. Counts are not runnable-program totals or an exhaustive count of global libraries. Live checks on 19/10/2026 retrieved 106 matching JavaScript blocks for Axios and four matching CMake blocks; those retrievals do not prove every block runs.

Public source URLs

Load public HTTPS documentation, repository README files or recognised raw source files. Private/local addresses, embedded credentials, nonstandard ports and unsafe redirects are rejected by public-source fetching. Pages are data, not assistant instructions. Authenticated or client-rendered documentation may need another public usage page.

Assembling and understanding code

Preserve, explain, add and test

Preview before adding

Selecting a source example shows its code and explanation without changing the editor. Add snippet to code appends to the current program; Replace code with snippet is a separate action that protects unsaved edits. Undo update restores previous code and its guidance. New request clears discovery fields and results while retaining code. Clear code and closing protect unsaved edits.

Detailed explanations

Each preview describes purpose, setup, usage steps, observable operations, expected output where supplied, extension points and provenance. HTML/Markdown extraction retains nearby publisher context where available. Python parsing identifies top-level imports, functions and classes. Recognised operations explain event handlers, DOM creation/lookup, HTTP requests, file access, output, input, asynchronous work and queries. Unrecognised fragments state that their behaviour cannot be reliably inferred; source context is not replaced by invented semantics.

Python as teaching reference

All 22 selector choices have a language/tool guide. Comparisons describe practical differences such as indentation versus braces, declarations, equality, null values, arrays and indexes, pipelines, SQL set operations, markup and build configuration. Python is a useful teaching reference; language-specific semantics and toolchains remain necessary. A syntax checker does not translate one language into another.

Assembly limits

Appending preserves existing text. It does not automatically connect variables, resolve imports, merge multiple entry points, remove duplicate declarations, complete fragments or satisfy a feature request. Review execution order, types, names, dependencies and source version. Define functions before calling them; insert calls where the application flow needs them. Check syntax and expected behaviour after each meaningful addition.

Save and reopen

Save code exports source with the displayed code language extension. Project > Save project with explanations writes a versioned JSON document with code, language, explanation, request and origin/source references. Open project validates that format and restores editable content without execution. Save all examples exports matching source blocks and their references. There is no automatic session recovery: save before closing.

Syntax checks and execution

Explicit tests with visible diagnostics

Syntax checking

Python uses ast.parse and reports line, column, original text, a caret and an explanation for common colon, indentation, bracket and string errors. Installed Node.js (--check), Ruby (-c), PHP (-l), Bash (-n) and TypeScript (--noEmit) provide their own checks. TypeScript checks types too. Other languages receive their documented compiler/project workflow; HTML/CSS have no built-in standards validator. A grammar pass does not establish correct imports, values, file access or program behaviour.

Runtime diagnostics

Run output preserves original stdout/stderr and exit status. Recognised diagnostics explain syntax errors, unexpected tokens/end of input, undefined names, missing modules/files, type errors, indentation and TypeScript errors. Explanations suggest checks and include relevant Python comparisons; they are not guaranteed root-cause diagnoses or silently applied edits.

Local runtime adapters

When the relevant executable is installed on PATH, Python, Node.js JavaScript, Ruby, PHP, PowerShell, Bash, R, Go, Dart, Swift and F# can run as child processes. TypeScript has a check adapter but needs compilation in its project before execution. C/C++, C#, Java, Kotlin, Scala, Rust, SQL and CMake receive compiler/project instructions. SQL needs the correct engine/schema; CMake configures a build rather than being the compiled program.

Run controls and permissions

Run code requires explicit confirmation of the currently reviewed editor source. Arguments are a JSON list of strings, passed without a command shell. Choose a working folder for relative input/output paths. Source files are created in a temporary run directory. Standard input can be sent from Run output. A 30-second limit, 1 MB output limit and Stop terminate the child process; Windows requests termination of its process tree. Temporary run files are cleaned after execution. Programs run with the user account permissions, including file and network access: this is not a sandbox.

Web preview

JavaScript using document/window and HTML/CSS open in a browser preview. DOM JavaScript is wrapped in an HTML page with an error display. Example fragments may still need their original markup and project. A preview launch is not proof that a page behaves correctly. Browser processes and their lifetime are separate from the child-process limits; use browser diagnostics and visible interaction checks.

Models, caching and operating limits

Optional services and explicit proof boundaries

Models / API

Discovery covers recognised local service listener ports, OLLAMA_HOST and saved localhost service URLs. Ollama /api/tags, LM Studio native/legacy lists and compatible /v1/models are supported. Connection is explicit; supported LM Studio APIs can load the selected downloaded language model. Embedding models are listed but rejected for generation. Service tokens are session-only. Model weights, credentials and serving runtimes are not redistributed.

Generation boundary

Selected Ollama /api/generate or compatible /v1/chat/completions receives the request, selected library metadata, up to three matching examples, current code and expansion context. Context over 24,000 characters is rejected without truncation. Empty/truncated responses and unchanged updates are rejected. Python syntax and named-component retention are checked, with a limited repair attempt. Local-model behaviour and hardware requirements vary; those checks do not prove all requested features. Generation and source retrieval never automatically execute code.

Caching and network limits

The per-user .og-companion folder stores model service URLs, monitor placement, document JSON, compressed PyPI names and collections. Fresh document/name caches are reused for 24 hours; stale retained data can survive refresh failure. Offline mode uses supplied/cached material and reports uncached sources. Registry requests normally allow 25 seconds and 12 MB; the PyPI name index allows 60 seconds and 160 MB. Documentation text allows 25 seconds and 8 MB; collection databases allow 160 MB. UI network work runs in background workers and cleared requests discard late results.

Foundations

CSV-to-JSON reads real headers, rejects missing/blank/extra cells, reports accepted/rejected rows, writes JSON and protects the input from overwrite. The file-integrity starter records SHA-256 hashes in 65,536-byte chunks, avoids symbolic-link files, reports read errors, refuses baseline overwrite and compares added/removed/changed relative paths. Protect the baseline; hashes identify changes rather than proving file safety. Tests exercise controlled files and error paths.

Privacy, licences and acceptance

Online sources receive search terms; model services receive the selected request/code and determine where inference runs. Source URLs, publisher attribution and available licence information accompany examples. This is not an automatic licence or vulnerability audit. No automatic package installation, sandbox, analytics or signing certificate is supplied. Other operating-system builds, all language runtimes, a clean second computer, live LM Studio and arbitrary downloaded/model-generated code remain separate acceptance checks. Product and investor portfolios prepared for version 2.2 need revision before describing version 2.3.

Built-in basics and hosted API models

OG Code Companion 2.4.0 | 19/10/2026

Basics before AI

All 22 language/tool choices now have a supplied baseline example. Common languages include Python, JavaScript, TypeScript, C, C++, C#, Java, HTML/CSS, SQL, Go, Rust, PHP, Ruby, PowerShell and Bash; R, Swift, Kotlin, Dart, F#, Scala and CMake also have basics. C includes stdio.h and string.h examples. The catalogue totals 55 snippets across 22 choices, plus the two existing complete Python foundations. These counts describe supplied content, not a promise that every language runtime has been executed.

Immediate language refresh

Choosing a language refreshes its supplied examples without replacing existing editor code. An empty package search restores those basics. Baseline example loading does not make a network request. Preview, Add snippet, explanations, syntax comparisons, undo and project saving remain available without a model. Local or hosted models can expand the assembled program when explicitly requested.

Add an API model

Open Models / API, choose Add API model and supply the provider/protocol, API base URL, exact model ID and API key. Presets cover OpenAI Responses, Anthropic Messages and Gemini through its documented OpenAI-compatible endpoint. Custom services can implement Chat Completions or Responses, with Bearer, api-key or x-api-key authentication. Other proprietary protocols and model/account restrictions can require another adapter. Local Ollama and LM Studio discovery remains available.

Credentials, data and billing

Keys exist only in process memory and are excluded from settings, project files and shared packages. Forget API keys removes the configured session credentials. Configuring/selecting an API model sends no request and is labelled unverified. Generation and expansion explicitly send the task and current code to that provider; its charges and data policies apply. Hosted endpoints require HTTPS, contain no embedded credentials/query parameters, and redirects are rejected so keys are not forwarded to another destination.

Contracts and verification

og_companion_providers.py implements the hosted protocols, session credential registry, response extraction and redacted network errors. Responses have a 2 MB limit and a 300-second timeout. Network failures are not automatically retried; the existing source-validation loop can make one correction request after an invalid generated result. Tests cover request schemas, authentication, key exclusion/forgetting, redirects, error redaction, truncated output and expansion context. No live hosted generation was performed with a real key; recipient account access and model quality remain unverified.

Provider references

OpenAI: https://developers.openai.com/api/docs/guides/text ; Anthropic: https://platform.claude.com/docs/en/api/messages/create ; Gemini: https://ai.google.dev/gemini-api/docs/openai . The product/technical and investor portfolios prepared for 2.2 still need revision before describing 2.4.